Reverse Engineering



Perché si effettua ?

- Per analisi a causa della documentazione mancante
- Avere accesso a piattaforme proprietarie
- Controlli di sicurezza
- Curiosità

Risoluzione CTF

È necessario comprendere come funziona un programma, ma senza avere accesso al codice sorgente.
Il processo può richiedere un pò di tempo

Nella realtà:

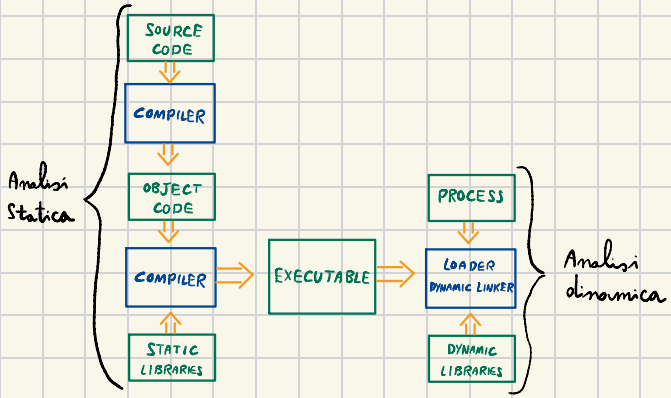
È utile per analizzare malware, rimuovere ramsomware
Ottenere gratuitamente software proprietari, ecc.



Punti Chiave

- Valutazione delle vulnerabilità
- Analisi dei malware
- Pwning
- Algorithm Reversing
- Hacking dispositivi embedded

Lifecycle di un Programma



Eseguibili

- Ogni S.O. ha il suo formato specifico:
  - ELF (*NIX), PE (Windows), Mach-O (MacOS, iOS)
- Generalmente si usa lo stesso formato sia per i programmi che per le librerie
- Un eseguibile è costituito da 3 parti, mappate in memoria:
  - .text, (ro)data, .bss
- Per ogni formato c'è un import specifico per le DLL:
  - GOT/PLT (ELF), IAT (PE)
- Metodi di loading:
  - Fixed Address
  - Relocation
  - Position-independent

Executable and Linked Format (ELF)

- Formato usato nei sistemi Unix-Like
- Disegnato per essere flessibile, estendibile e cross-platform
- Program Headers descrivono i segmenti (Virtual Mapping)
- Section Headers descrivono le sezioni e come caricarle nei segment
- Supporta la Relocation

Tecniche e Tools per la Reverse Engineering

- **Analisi Statica** non avvia l'eseguibile
  - disassembly, decompilation
  - abstract interpretation
  - symbolic execution
- **Analisi Dinamica** avvia l'eseguibile
  - debugging
  - dynamic binary instrumentation

Tools:

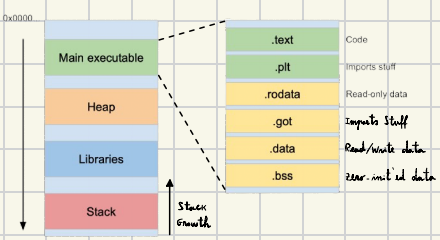
- IDA: gui, industry standard
- RADARE2: CLI, open source
- GHIDRA: NSA reversing tool (open)
- Binary Ninja, Objdump
- HEX Editor: Patch programs, inspect file formats change content of file

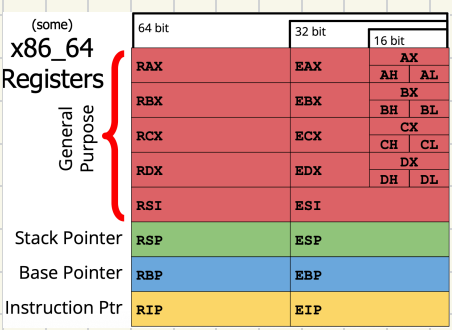
Altri Tools

- Executable Informations: file, readelf, PEview, ...
- Useful Commands: strings, ptrace, ltrace
- Debuggers: gdb, WinDbg, OllyDbg, Immunity Debugger, qira, ...
- Decompilers

ASSEMBLY

Linux Process Memory





Istruzioni:

- MOV <dst>, <src>: copia SRC → DST
- LEA <dst>, <src>: load effective address of SRC → DST
- PUSH <src>: Decrementa RSP, inserisce SRC → stack
- POP <dst>: inserisce stack → DST, incrementa RSP
- NOP: non fa nulla, utile nel patching per rimuovere CALL, CMP
- ADD <dst>, <src>: DST += SRC
- SUB <dst>, <src>: DST -= SRC
- XOR <dst>, <src>: DST = DST ^ SRC
- CALL <dst>: chiama subroutine DST
- RET: return da subroutine

Flags

- registro che contiene 1-bit flags:
  - es: OF, SF, ZF, AF, PF e CF
- ZF = Zero flag: 1 se il risultato dell'ultima operazione è 0
- SF = Sign flag: 1 se il risultato dell'ultima operazione è < 0

Istruzioni Condizionate (Basate sulla flag)

- CMP <dst>, <src>: DST - SRC, setta flag, butta via il risultato
- JMP <dst>: salta all'indirizzo DST
- JXX <dst>:
  - JZ / JE → JMP se ZF = 1
  - JNZ / JNE → JMP se ZF = 0
  - JB, JA → JMP se DST Below/Above SRC (unsigned)
  - JL, JG → JMP se DST </> SRC (signed)

Passaggio Parametri a Subroutine

In x86 ci sono più convenzioni per passare i parametri:
- tramite STACK
- tramite registri

I valori di return vengono passati tramite registri RAX/EAX